

Dakar Institute of Technology



Habilitation n°3386-21 FEV.2024

Domaine : Sciences et Technologies

Département : Informatique

Spécialité : Intelligence Artificielle

Master 2 IA : Examen Outils de Versioning

Projet : Bibliothèque Numérique avec Système de Recommandation

Présenté par

D. M. Maurice AHOUANSON

M. DIAKITE

Abdoul G. DIALLO

Ibrahima DIALLO

Seydou KABORE

Alassane KANE

Abdin KOUSSUBE

Fatoumata Bintou SIDIBE

Année Académique : 2025 – 2026

Table des matières

1	Présentation du projet	4
1.1	Contexte et objectifs	4
1.2	Problématique	4
1.3	Objectifs du projet	5
1.4	Structure du rapport	5
2	Architecture du Système	6
2.1	Vue d'ensemble de l'architecture microservices	6
2.2	Fonctionnalités principales	6
2.3	Stack technique	7
2.4	Comptes de démonstration	7
2.4.1	Services et ports	8
2.5	Réseau Docker Compose	8
2.6	Flux d'emprunt	9
2.7	Schéma de base de données	11
2.8	Règles métier	11
3	Description des APIs	12
3.1	Service Livres — port 3001	12
3.2	Service Utilisateurs — port 3002	12
3.3	Service Emprunts — port 3003	13
3.4	Service Recommandation — port 3004	13
3.5	Codes de retour HTTP	13
3.6	Exemple de session curl	14
4	Workflow Git utilisé	14
4.1	Stratégie de branches	14
4.2	Cycle de contribution	15
4.3	Conventions de commit	15
4.4	Pipeline CI/CD avec Jenkins	16
5	Pipeline DVC	16
5.1	Présentation de DVC	16
5.2	Schéma des dépendances	16
5.3	Définition du pipeline (<code>dvc.yaml</code>)	18
5.4	Hyperparamètres du modèle	18
5.5	Commandes DVC principales	19
6	Métriques du modèle	19

6.1	Algorithme SVD	19
6.2	Métriques d'évaluation	20
6.3	Stratégie de recommandation multi-niveaux	20
6.4	Déploiement d'un nouveau modèle	20
7	Captures d'écran de l'application	21
7.1	Catalogue public	21
7.2	Authentification	22
7.3	Catalogue administrateur	23
7.4	Tableau de bord gestionnaire	23
7.5	Gestion des emprunts	24
7.6	Gestion des utilisateurs	24
7.7	Système de recommandation	25
	Conclusion	25

Résumé

Le présent projet, intitulé **Bibliothèque Numérique DIT**, porte sur la conception et la mise en œuvre d'une plateforme complète de gestion de bibliothèque académique basée sur une architecture microservices. L'objectif principal était de développer un système moderne permettant la gestion des livres, des utilisateurs et des emprunts, enrichi d'un système de recommandation intelligente basé sur l'algorithme SVD (Singular Value Decomposition), le tout versionné et orchestré selon les meilleures pratiques DevOps et MLOps.

L'architecture développée repose sur cinq microservices indépendants : un service de gestion des livres, un service d'authentification des utilisateurs, un service de gestion des emprunts, un service de recommandation basé sur FastAPI et scikit-surprise, et une interface frontend en React 18. Chaque service dispose de son propre Dockerfile et communique via des API REST. L'ensemble est orchestré par Docker Compose avec une base de données PostgreSQL 17 partagée.

Le versioning des données et du modèle ML a été assuré avec DVC (Data Version Control), permettant un pipeline reproductible en trois étapes : prétraitement, entraînement et évaluation. Un remote Google Drive a été configuré pour stocker les artefacts. Le modèle SVD a été amélioré entre deux versions, passant d'un RMSE de 1,42 à 1,38, soit une amélioration de 4%.

Le versioning du code a suivi le workflow Gitflow avec des branches dédiées, des tags sémantiques et des commits conventionnels. Un pipeline CI/CD a été déployé avec Jenkins pour automatiser les étapes de construction, de test et de déploiement.

Ce projet démontre la pertinence des architectures microservices modernes appliquées à la gestion académique, et constitue une application concrète des compétences en développement backend, conteneurisation, machine learning et gestion de versions.

1 Présentation du projet

1.1 Contexte et objectifs

À l’heure du numérique, les institutions académiques génèrent quotidiennement un volume croissant de données liées à leurs activités : gestion des ressources documentaires, suivi des emprunts, analyse des comportements des étudiants. Dans le secteur de la gestion de bibliothèque, chaque emprunt, retour ou consultation représente une source précieuse d’informations pouvant être exploitée pour améliorer les services offerts aux utilisateurs.

Face à cette réalité, les technologies modernes de développement web et de machine learning offrent aujourd’hui des solutions performantes pour concevoir des plateformes intelligentes, capables non seulement de gérer les opérations courantes, mais aussi de proposer des recommandations personnalisées basées sur l’historique des utilisateurs.

C’est dans ce contexte que s’inscrit le projet **Bibliothèque Numérique DIT**, dont l’objectif est de concevoir une plateforme web complète de gestion de bibliothèque académique, construite sur une architecture microservices, enrichie d’un système de recommandation basé sur l’apprentissage automatique, et soumise à une gestion avancée du versioning. À travers cette réalisation, il s’agit de mettre en pratique les concepts fondamentaux des outils de versioning — Git, Docker et DVC — dans un contexte applicatif complet et professionnel.

1.2 Problématique

Le Dakar Institute of Technology (DIT) gère jusqu’à présent sa bibliothèque académique de façon entièrement manuelle. Cette approche engendré de nombreuses lacunes opérationnelles :

- ✓ **Difficulté de suivi des livres** : absence de visibilité en temps réel sur la disponibilité des ouvrages ;
- ✓ **Absence de statistiques fiables** : impossibilité de produire des indicateurs sur les emprunts ou les retards ;
- ✓ **Gestion inefficace des emprunts** : calcul manuel des pénalités, risques d’erreurs ;
- ✓ **Manque d’accès numérique** : les étudiants ne peuvent pas consulter le catalogue ou gérer leurs emprunts à distance.

La problématique centrale de ce projet peut être formulée ainsi : *comment concevoir une plateforme web moderne à architecture microservices permettant de gérer efficacement une bibliothèque académique, d’automatiser les emprunts, de personnaliser les recommandations par machine learning, et de garantir la reproductibilité et la traçabilité du code, des données et des modèles ?*

1.3 Objectifs du projet

L'objectif général de ce projet est de développer une plateforme numérique complète de gestion de bibliothèque académique reposant sur une architecture microservices, intégrant un système de recommandation ML et une gestion avancée du versioning.

Spécifiquement, il s'agit de :

1. Développer cinq microservices indépendants (livres, utilisateurs, emprunts, recommandation, frontend) communiquant via des API REST ;
2. Implémenter un système de recommandation basé sur l'algorithme SVD (*Singular Value Decomposition*) à partir de l'historique des emprunts ;
3. Conteneuriser l'ensemble de l'application avec Docker et orchestrer les services avec Docker Compose ;
4. Versionner le code avec Git selon le workflow Gitflow, et les données/modèles avec DVC ;
5. Mettre en place un pipeline CI/CD avec Jenkins pour automatiser construction, tests et déploiement.

1.4 Structure du rapport

Le présent rapport est structuré en plusieurs chapitres complémentaires. La **Section 1** présente les généralités, le contexte, la problématique et les objectifs. La **Section 2** décrit l'architecture du système, le schéma microservices et la structure du projet. La **Section 3** présente la description détaillée des APIs de chaque service. La **Section 4** expose le workflow Git avancé utilisé. La **Section 5** décrit le pipeline DVC avec ses schémas de dépendances et ses métriques. Enfin, la **Section 6** présente la réalisation concrète du projet avec les captures d'écran de l'application.

2 Architecture du Système

2.1 Vue d'ensemble de l'architecture microservices

La plateforme Bibliothèque Numérique DIT repose sur une architecture microservices composée de six conteneurs Docker orchestrés par Docker Compose. Chaque service expose une API REST indépendante. La Figure 1 présente le schéma conceptuel de l'architecture globale.

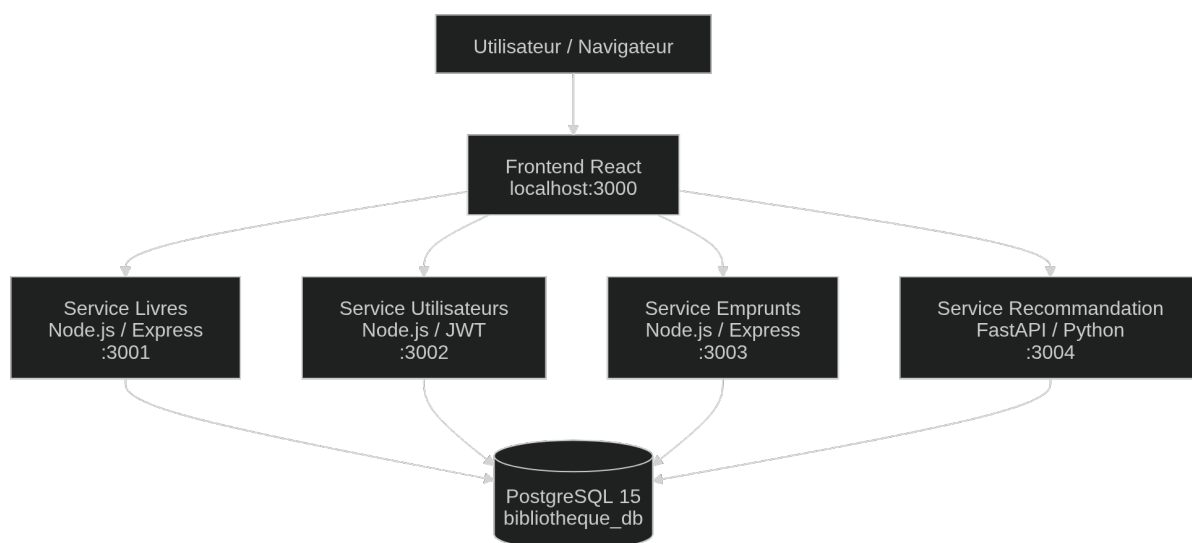


FIGURE 1 – Architecture microservices de la Bibliothèque Numérique DIT

2.2 Fonctionnalités principales

- **Catalogue de livres** : 150+ ouvrages techniques (IA, Data Science, DevOps, NLP, Robotique, Génie Logiciel, etc.), avec recherche multicritères.
- **Gestion des utilisateurs** : inscription, authentification JWT (24 h), trois rôles — étudiant, professeur, personnel (gestionnaire).
- **Système d'emprunts** : durée de 30 jours, limite de 3 emprunts simultanés par utilisateur, détection automatique des retards, pénalités à 500 FCFA/jour.
- **Recommandations personnalisées** : algorithme SVD (filtrage collaboratif) via `scikit-surprise`, avec fallback catégoriel et barre de progression.
- **Tableau de bord gestionnaire** : KPIs temps réel (livres disponibles, emprunts, retards, pénalités FCFA), panneau latéral d'inspection.
- **Pipeline ML versionné** : prétraitement, entraînement et évaluation reproductibles

via DVC.

- **CI/CD automatisé** : pipeline Jenkins pour build, test, entraînement et déploiement.

2.3 Stack technique

TABLE 1 – Stack technique du projet Bibliothèque Numérique DIT

Composant	Technologie	Rôle
books-service	Node.js 20 · Express · PostgreSQL	CRUD catalogue livres
users-service	Node.js 20 · Express · JWT · bcrypt	Authentification & utilisateurs
loans-service	Node.js 20 · Express · PostgreSQL	Emprunts & pénalités
recommendation-service	Python 3.11 · FastAPI · scikit-surprise	SVD + fallback ML
api-gateway	Node.js 20 · Express	Routage inter-services
Frontend	HTML5 · CSS3 · JavaScript (vanilla)	Interface SPA (dark theme)
Base de données	PostgreSQL 17	Persistance relationnelle
Orchestration	Docker Compose (profils dev/prod)	7 conteneurs
Versionnage ML	DVC	Données, modèles et métriques
CI/CD	Jenkins	Build, test, déploiement

2.4 Comptes de démonstration

Rôle	Email	Mot de passe
Gestionnaire (Admin)	admin@dit.sn	admin2026
Étudiant	mamadou1@dit.sn	dit2026
Étudiant	fatou2@dit.sn	dit2026
Professeur	ibrahima3@dit.sn	dit2026

TABLE 2 – Comptes de démonstration disponibles après initialisation

2.4.1 Services et ports

Service	Port	Technologie	Profil
Frontend nginx	8080	HTML · CSS · JS · nginx :alpine	prod
API Gateway	3000	Node.js · Express	dev + prod
Service Livres	3001	Node.js · Express	dev + prod
Service Utilisateurs	3002	Node.js · Express · JWT	dev + prod
Service Emprunts	3003	Node.js · Express	dev + prod
Service Recommandation	3004	Python · FastAPI	dev + prod
PostgreSQL 17	5432	Base de données	dev + prod
pgAdmin	8081	Interface admin DB	dev

TABLE 3 – Services Docker, ports exposés et profils Docker Compose

2.5 Réseau Docker Compose

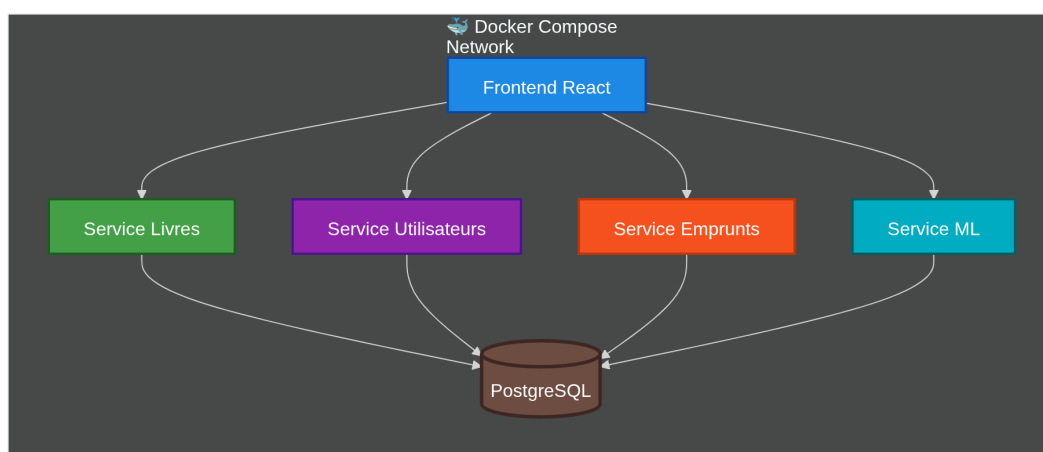


FIGURE 2 – Réseau Docker Compose — interconnexion des conteneurs sur un réseau interne partagé

Le fichier `docker-compose.yml` définit deux profils :

- **dev** : inclut PostgreSQL, pgAdmin, API Gateway et les quatre services. Permet le débogage via pgAdmin (`localhost:8081`).
- **prod** : remplace pgAdmin par le frontend nginx, prêt pour la production.

Listing 1 – Lancement des environnements

```
# Environnement de développement
docker compose --profile dev up --build -d

# Environnement de production (avec frontend)
docker compose --profile prod up --build -d
```

```
# Verification de l'etat des conteneurs  
docker compose --profile prod ps
```

2.6 Flux d'emprunt

La Figure 3 illustre le processus de gestion des emprunts dans la plateforme de bibliothèque numérique. Lorsqu'un utilisateur demande un livre, le service d'emprunts vérifie d'abord la disponibilité de l'ouvrage. Si le livre est disponible, le système contrôle ensuite que l'utilisateur possède moins de trois emprunts actifs. Si toutes les conditions sont respectées, l'emprunt est validé et enregistré dans la base de données PostgreSQL ; sinon, la demande est refusée.

Ce service applique également les règles métier : 3 emprunts simultanés maximum, durée de 30 jours, pénalité de 500 FCFA par jour de retard.

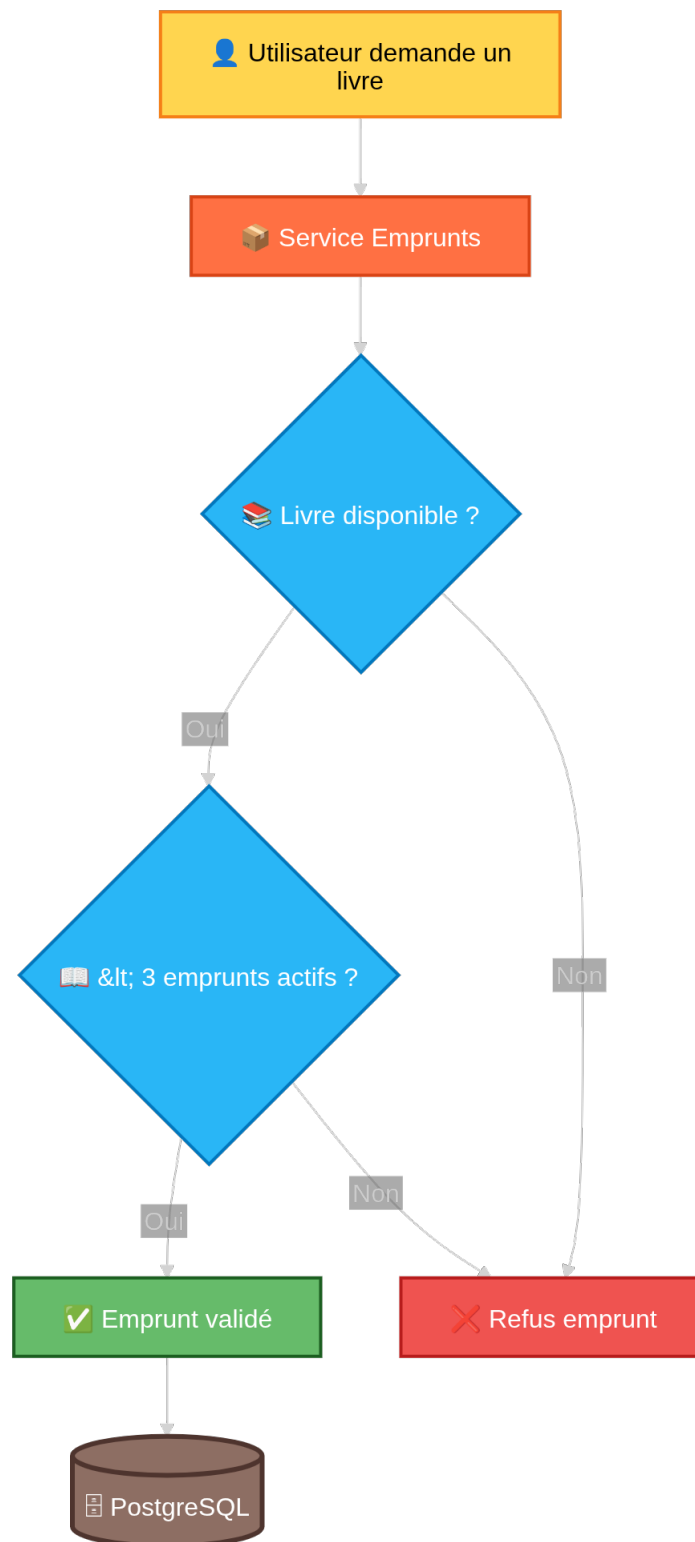


FIGURE 3 – Diagramme de flux — validation d’une demande d’emprunt

Chaque demande d’emprunt est soumise à deux vérifications séquentielles :

1. Le livre possède-t-il des exemplaires disponibles ?
2. L’utilisateur a-t-il moins de 3 emprunts actifs ?

Si les deux conditions sont satisfaites, l'emprunt est créé et la disponibilité décrémentée en base. Sinon, une erreur 409 `Conflict` est retournée.

2.7 Schéma de base de données

La base `dit_bibliotheque` (PostgreSQL 17) comprend quatre tables :

utilisateurs		livres	
<code>id</code>	UUID PK	<code>id</code>	UUID PK
<code>nom</code>	VARCHAR(100)	<code>titre</code>	VARCHAR
<code>prenom</code>	VARCHAR(100)	<code>auteur</code>	VARCHAR
<code>email</code>	VARCHAR UNIQUE	<code>isbn</code>	VARCHAR UNIQUE
<code>mot_de_passe</code>	VARCHAR(255)	<code>categorie</code>	VARCHAR
<code>type_utilisateur</code>	VARCHAR(20)	<code>nombre_exemplaires</code>	INT
<code>model_id</code>	VARCHAR(10) UNIQUE	<code>exemplaires_disponibles</code>	INT
<code>actif</code>	BOOLEAN	<code>model_id</code>	VARCHAR UNIQUE
<code>created_at</code>	TIMESTAMP	<code>created_at</code>	TIMESTAMP

TABLE 4 – Tables utilisateurs et livres

emprunts		notes	
<code>id</code>	UUID PK	<code>id</code>	UUID PK
<code>utilisateur_id</code>	UUID FK	<code>utilisateur_id</code>	UUID FK
<code>livre_id</code>	UUID FK	<code>livre_id</code>	UUID FK
<code>date_emprunt</code>	DATE	<code>note</code>	NUMERIC(2,1)
<code>date_retour_prevue</code>	DATE	<code>created_at</code>	TIMESTAMP
<code>date_retour_effective</code>	DATE	UNIQUE(<code>utilisateur_id</code> , <code>livre_id</code>)	
<code>statut</code>	VARCHAR		

TABLE 5 – Tables emprunts et notes — le statut prend les valeurs `en_cours`, `en_retard` ou `retourne`

2.8 Règles métier

Durée d'emprunt : 30 jours (configurable via `DUREE_EMPRUNT_JOURS`)

Limite : 3 emprunts actifs maximum par utilisateur (erreur 409 si dépassé)

Pénalité : jours de retard $\times$ 500 FCFA (ex. : 5 jours = 2 500 FCFA)

Détection retards : automatique via `POST /api/emprunts/detecter-retards`, appelé à chaque chargement du dashboard

Création / retour : gestionnaire uniquement (JWT `type=personnel`)

3 Description des APIs

3.1 Service Livres — port 3001

Méthode	Endpoint	Description
GET	/api/livres	Liste paginée (recherche titre/auteur/ISBN/catégorie)
GET	/api/livres/disponibles	Livres avec stock > 0
GET	/api/livres/:id	Détail d'un livre
POST	/api/livres	Créer un livre (<i>Admin</i>)
PUT	/api/livres/:id	Modifier un livre (<i>Admin</i>)
DELETE	/api/livres/:id	Supprimer un livre (<i>Admin</i>)
GET	/api/admin/livres/stats	Statistiques catalogue (<i>Admin</i>)
POST	/api/admin/livres/bulk	Import en masse (<i>Admin</i>)

TABLE 6 – Endpoints du service Livres

3.2 Service Utilisateurs — port 3002

Méthode	Endpoint	Description
POST	/api/auth/register	Inscription (public)
POST	/api/auth/login	Connexion → token JWT (public)
GET	/api/utilisateurs/stats	Répartition par type (public)
GET	/api/utilisateurs/me	Profil de l'utilisateur connecté (JWT)
GET	/api/utilisateurs	Liste paginée (JWT)
GET	/api/utilisateurs/:id	Détail utilisateur (JWT)
PUT	/api/utilisateurs/:id	Modifier (JWT)
PATCH	/api/utilisateurs/:id/toggle	Activer / désactiver (JWT)
DELETE	/api/utilisateurs/:id	Supprimer (JWT)

TABLE 7 – Endpoints du service Utilisateurs

3.3 Service Emprunts — port 3003

Méthode	Endpoint	Description
GET	/api/emprunts	Liste paginée (public)
GET	/api/emprunts/stats	Statistiques globales (public)
GET	/api/emprunts/penalites	Retards + pénalités FCFA (public)
GET	/api/emprunts/export/csv	Export CSV pour DVC (public)
GET	/api/emprunts/utilisateur/:id	Historique d'un utilisateur (public)
GET	/api/emprunts/:id	Détail d'un emprunt (public)
POST	/api/emprunts	Créer un emprunt (<i>Admin</i>)
PATCH	/api/emprunts/:id/retourner	Valider le retour (<i>Admin</i>)
POST	/api/emprunts/detecter-retards	Mise à jour des statuts (<i>Admin</i>)
GET	/api/admin/dashboard	Tableau de bord complet (<i>Admin</i>)

TABLE 8 – Endpoints du service Emprunts

3.4 Service Recommandation — port 3004

Méthode	Endpoint	Description
GET	/api/recommendations/:user_id	Recommandations (UUID ou model_id)
POST	/api/train	Ré-entraîner le modèle SVD
GET	/api/model/info	Informations sur le modèle chargé

TABLE 9 – Endpoints du service Recommandation

3.5 Codes de retour HTTP

Code	Signification
200	Succès
201	Ressource créée
204	Supprimé (sans corps de réponse)
400	Données invalides ou manquantes
401	Token manquant ou invalide
403	Droits insuffisants (non-gestionnaire)
404	Ressource introuvable
409	Conflit (doublon ISBN, limite d'emprunts dépassée. ...)
503	Modèle ML non chargé

TABLE 10 – Codes de retour HTTP standardisés

3.6 Exemple de session curl

Listing 2 – Authentification, emprunt et retour via curl

```
# 1. Obtenir un token JWT (gestionnaire)
TOKEN=$(curl -s -X POST http://localhost:3002/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"admin@dit.sn","mot_de_passe":"admin2026"}' \
| python3 -c "import sys,json; print(json.load(sys.stdin)['token'])")

# 2. Lister les livres disponibles
curl "http://localhost:3001/api/livres/disponibles"

# 3. Creer un emprunt (gestionnaire)
curl -X POST http://localhost:3003/api/emprunts \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TOKEN" \
-d '{"utilisateur_id":"<uuid>","livre_id":"<uuid>"}'

# 4. Valider un retour
curl -X PATCH http://localhost:3003/api/emprunts/<id>/retourner \
-H "Authorization: Bearer $TOKEN"

# 5. Test de securite : route protegee sans token -> 401
curl -s -o /dev/null -w "%{http_code}" \
-X POST http://localhost:3003/api/emprunts \
-H "Content-Type: application/json" \
-d '{"utilisateur_id":"x","livre_id":"y"}'

# 6. Route publique -> 200
curl -s -o /dev/null -w "%{http_code}" \
http://localhost:3003/api/emprunts/stats
```

4 Workflow Git utilisé

4.1 Stratégie de branches

Le projet adopte une stratégie **GitFlow** adaptée à une équipe de 8 personnes :

- **main** : branche de production, toujours déployable, protégée.
- **develop** : branche d'intégration continue, cible des Pull Requests.
- **feature/<nom>** : une branche par fonctionnalité ou microservice (ex. **feature/loans-penalties**, **feature/recommendation-svd**).

- `fix/<nom>` : corrections de bugs isolées.
- `dvc/pipeline` : modifications du pipeline DVC et des artefacts ML.

4.2 Cycle de contribution

Listing 3 – Workflow Git typique d'un développeur

```
# 1. Partir de develop a jour
git checkout develop
git pull origin develop
git checkout -b feature/loans-penalties

# 2. Developper et committer
git add backend/loans-service/src/
git commit -m "feat(loans): add penalty calculation at 500 FCFA/day"

# 3. Pousser et ouvrir une Pull Request vers develop
git push origin feature/loans-penalties
# -> Ouvrir la PR sur GitHub/GitLab, assignation d'un pair

# 4. Apres approbation : merge dans develop -> CI Jenkins se
    declenche

# 5. Livraison : merge develop -> main (release tag)
git checkout main
git merge develop
git tag -a v1.2.0 -m "Release 1.2.0 - systeme de penalites"
git push origin main --tags
```

4.3 Conventions de commit

Le projet suit le format **Conventional Commits** :

Préfixe	Usage
<code>feat:</code>	Nouvelle fonctionnalité
<code>fix:</code>	Correction de bug
<code>docs:</code>	Documentation (README, commentaires)
<code>refactor:</code>	Refactorisation sans changement de comportement
<code>test:</code>	Ajout ou modification de tests
<code>ci:</code>	Pipeline Jenkins ou DVC
<code>chore:</code>	Maintenance (dépendances, configuration)

TABLE 11 – Préfixes de commits conventionnels utilisés dans le projet

4.4 Pipeline CI/CD avec Jenkins

Le fichier `Jenkinsfile` à la racine définit les étapes suivantes, déclenchées automatiquement à chaque push sur `develop` ou `main` :

1. **Checkout** : récupération du code depuis Git.
2. **Build** : `docker compose --profile prod build` (reconstruction des images modifiées).
3. **Test** : health checks sur les 4 services API (`GET /health`).
4. **DVC Pipeline** : `dvc repro` — prétraitement, entraînement et évaluation du modèle SVD.
5. **Deploy** : `docker compose --profile prod up -d`.
6. **Notify** : notification de succès ou d'échec.

5 Pipeline DVC

5.1 Présentation de DVC

DVC (Data Version Control) [?] est un outil open-source de versionnage des données et des modèles ML. Il s'intègre à Git pour versionner les artefacts (données, modèles, métriques) sans les stocker dans le dépôt Git, et garantit la **reproductibilité** des expériences ML.

5.2 Schéma des dépendances

Le pipeline est défini dans `dvc.yaml` et comprend **3 étapes séquentielles** :

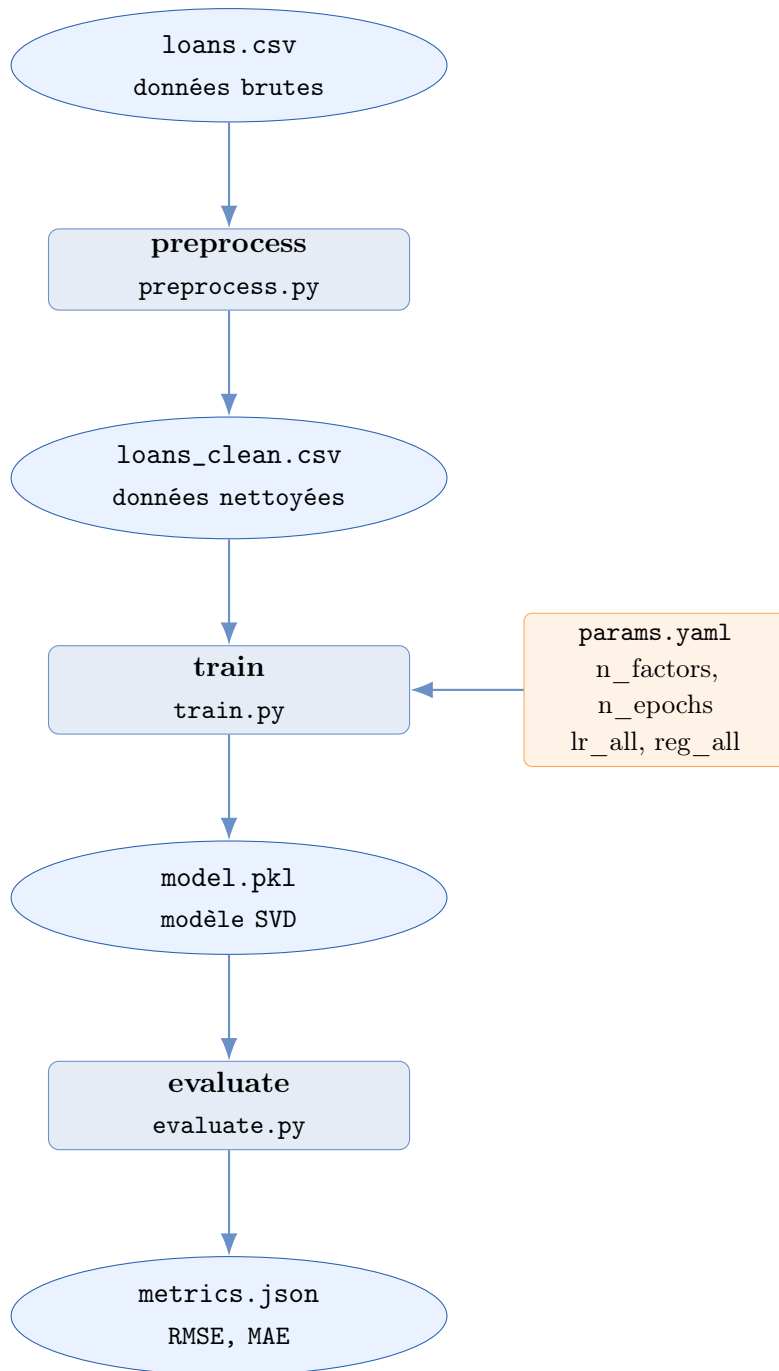


Figure — Schéma des dépendances du pipeline DVC (boîtes bleues : données/artefacts ; rectangles : étapes ; orange : hyperparamètres)

Étape	Commande	Entrée	Sortie
preprocess	preprocess.py	loans.csv	loans_clean.csv
train	train.py	loans_clean.csv + params.yaml	model.pkl
evaluate	evaluate.py	model.pkl + loans_clean.csv	metrics.json

TABLE 12 – Étapes, entrées et sorties du pipeline DVC

5.3 Définition du pipeline (dvc.yaml)

Listing 4 – Contenu complet du fichier dvc.yaml

```
stages:

  preprocess:
    cmd: python dvc-pipeline/preprocess.py
    deps:
      - dvc-pipeline/preprocess.py
      - dvc-pipeline/data/loans.csv
    outs:
      - dvc-pipeline/data/loans_clean.csv

  train:
    cmd: python dvc-pipeline/train.py
    deps:
      - dvc-pipeline/train.py
      - dvc-pipeline/data/loans_clean.csv
      - dvc-pipeline/params.yaml
    params:
      - dvc-pipeline/params.yaml:
          - train.n_factors
          - train.n_epochs
          - train.lr_all
          - train.reg_all
    outs:
      - dvc-pipeline/models/model.pkl

  evaluate:
    cmd: python dvc-pipeline/evaluate.py
    deps:
      - dvc-pipeline/evaluate.py
      - dvc-pipeline/models/model.pkl
      - dvc-pipeline/data/loans_clean.csv
    metrics:
      - metrics/metrics.json:
          cache: false
```

5.4 Hyperparamètres du modèle

Paramètre	Valeur	Description	Impact
n_factors	50	Dimensions latentes	Capacité du modèle
n_epochs	20	Itérations d'entraînement	Convergence
lr_all	0.005	Taux d'apprentissage	Vitesse de convergence
reg_all	0.02	Régularisation L2	Prévention du surapprentissage

TABLE 13 – Hyperparamètres SVD définis dans dvc-pipeline/params.yaml

5.5 Commandes DVC principales

Listing 5 – Workflow DVC complet

```
# Exporter les donnees depuis l'API Emprunts
curl http://localhost:3003/api/emprunts/export/csv \
  -o dvc-pipeline/data/loans.csv

# Reproduire le pipeline (reexecute seulement les etapes modifiees)
dvc repro

# Afficher les metriques
dvc metrics show

# Comparer avec la version precedente
dvc metrics diff HEAD~1
# Exemple de sortie :
# Path                               Metric   Old     New     Change
# metrics/metrics.json               rmse     1.42    1.38    -0.04
# metrics/metrics.json               mae      1.31    1.27    -0.04

# Versionner les artefacts
dvc add dvc-pipeline/models/model.pkl
dvc push

# Verifier l'etat du pipeline
dvc status
```

6 Métriques du modèle

6.1 Algorithme SVD

L'algorithme **SVD** (Singular Value Decomposition) de la bibliothèque `scikit-surprise` [?] est utilisé pour le filtrage collaboratif [?]. Il décompose la matrice utilisateur-livre en facteurs latents capturant les préférences implicites. La prédiction de la note $\hat{r}_{ui}$ est donnée par :

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u$$

où μ est la note globale moyenne, b_u et b_i sont les biais utilisateur et item, p_u et q_i sont les vecteurs latents.

6.2 Métriques d'évaluation

Les métriques sont calculées par `evaluate.py` via une validation croisée 5-fold et exportées dans `metrics/metrics.json` :

Métrique	Valeur	Définition	Interprétation
RMSE	1.38	$\sqrt{\frac{1}{n} \sum (r_{ui} - \hat{r}_{ui})^2}$	Erreur quadratique — pénalise les grandes erreurs
MAE	1.27	$\frac{1}{n} \sum r_{ui} - \hat{r}_{ui} $	Erreur absolue moyenne sur une échelle de 1 à 5

TABLE 14 – Métriques du modèle SVD (après entraînement sur les données d'emprunts DIT)

6.3 Stratégie de recommandation multi-niveaux

Scénario	Comportement
Utilisateur présent dans le modèle SVD	Filtrage collaboratif : prédiction des notes sur les livres non empruntés, retour des <code>top_k</code> meilleurs avec score affiché
≥ 5 emprunts, hors modèle SVD	Fallback catégoriel : livres populaires dans les catégories préférées de l'utilisateur (calculées depuis l'historique)
< 5 emprunts	Refus avec barre de progression affichant le nombre d'emprunts restants avant déblocage

TABLE 15 – Stratégie de recommandation adaptative

6.4 Déploiement d'un nouveau modèle

Listing 6 – Mise à jour du modèle dans le service de recommandation

```
# 1. Re-entraîner via DVC
dvc repro

# 2. Copier le modele dans le conteneur
docker cp dvc-pipeline/models/model.pkl \
  recommendation_service:/app/models/model.pkl

# 3. Recharger sans redemarrer le service
curl -X POST http://localhost:3004/api/train \
  -H "Content-Type: application/json" \
  -d '{"n_factors": 50, "n_epochs": 20, "lr_all": 0.005, "reg_all": 0.02}'

# 4. Verifier le chargement
```

```
curl http://localhost:3004/api/model/info
```

7 Captures d'écran de l'application

7.1 Catalogue public

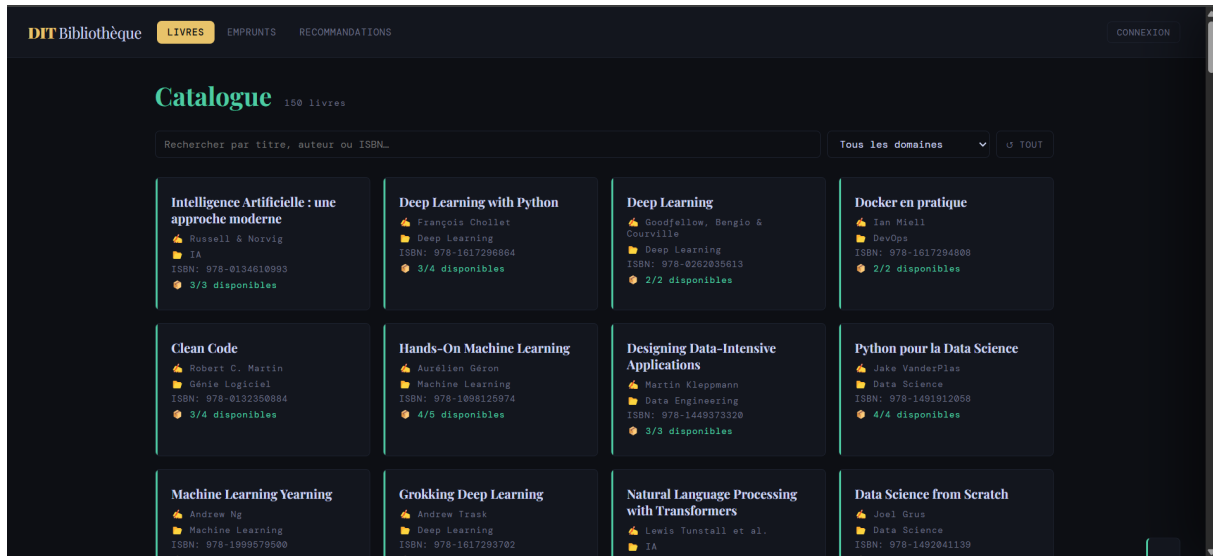
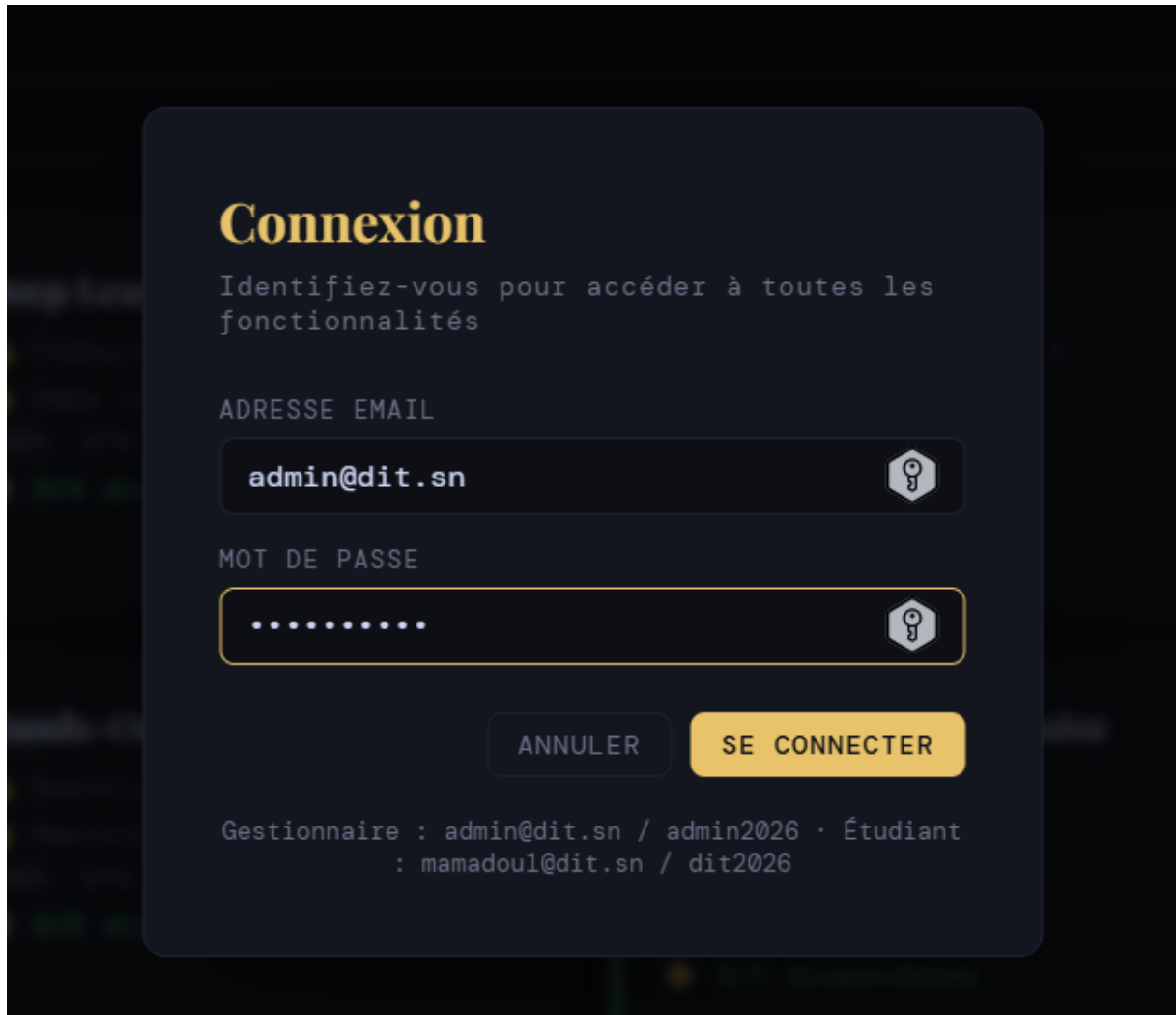


FIGURE 4 — Page d'accueil publique — 150 livres accessibles sans authentification avec recherche multicritères

7.2 Authentification

A dark-themed login modal with a title 'Connexion' in yellow. Below the title is a subtitle 'Identifiez-vous pour accéder à toutes les fonctionnalités' in a monospace font. There are two input fields: 'ADRESSE EMAIL' containing 'admin@dit.sn' and 'MOT DE PASSE' with masked characters. Both fields have a key icon on the right. At the bottom are 'ANNULER' and 'SE CONNECTER' buttons. A footer line shows user roles and credentials: 'Gestionnaire : admin@dit.sn / admin2026 · Étudiant : mamadou1@dit.sn / dit2026'.

Connexion

Identifiez-vous pour accéder à toutes les fonctionnalités

ADRESSE EMAIL

admin@dit.sn

MOT DE PASSE

.....

ANNULER SE CONNECTER

Gestionnaire : admin@dit.sn / admin2026 · Étudiant : mamadou1@dit.sn / dit2026

FIGURE 5 – Modal de connexion JWT — email et mot de passe, token valable 24 h

7.3 Catalogue administrateur

DIT Bibliothèque LIVRES UTILISATEURS EMPRUNTS DASHBOARD RECOMMANDATIONS DIT Admin PERSONNEL DÉCONNEXION

Catalogue 150 livres

Ajouter un livre

TITRE *	AUTEUR *	ISBN *	CATÉGORIE	EXEMPLAIRES
Clean Code	Robert Martin	978-0132350884	IA, DevOps...	1

+ AJOUTER

Rechercher par titre, auteur ou ISBN...

Tous les domaines ▼ 3 TOUT

Intelligence Artificielle : une approche moderne
Russell & Norvig
IA
ISBN: 978-0134610993
3/3 disponibles
SUPPRIMER

Deep Learning with Python
François Chollet
Deep Learning
ISBN: 978-1617296864
3/4 disponibles
SUPPRIMER

Deep Learning
Goodfellow, Bengio & Courville
Deep Learning
ISBN: 978-0262035613
2/2 disponibles
SUPPRIMER

Docker en pratique
Ian Miell
DevOps
ISBN: 978-1617294808
2/2 disponibles
SUPPRIMER

FIGURE 6 — Vue gestionnaire — formulaire d’ajout de livre visible à droite (champs : titre, auteur, ISBN, catégorie, exemplaires)

7.4 Tableau de bord gestionnaire

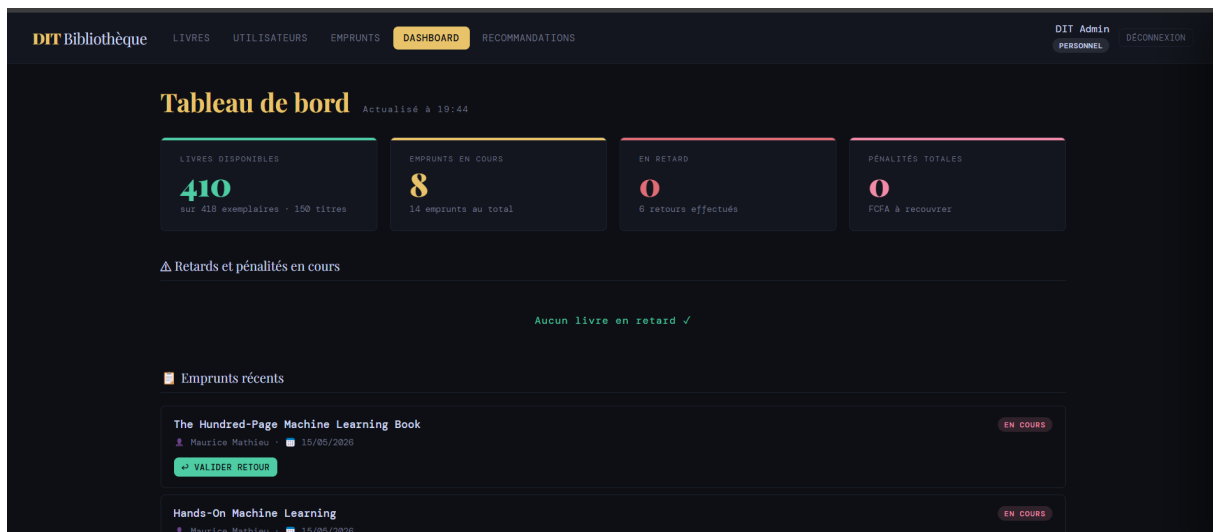


FIGURE 7 — Dashboard KPIs — 410 livres disponibles · 8 emprunts en cours · 0 retard · 0 FCFA de pénalités

7.5 Gestion des emprunts

The screenshot shows the 'Emprunts' section of the DIT Bibliothèque application. At the top, there's a navigation bar with 'DIT Bibliothèque' and links for 'LIVRES', 'UTILISATEURS', 'EMPRUNTS' (highlighted), 'DASHBOARD', and 'RECOMMANDATIONS'. On the right, there's a user profile 'DIT Admin' with 'PERSONNEL' and 'DÉCONNEXION' options.

The main heading is 'Emprunts' with a subtext '8 emprunts en cours'. Below this is a form titled 'Valider un emprunt'. It has two input fields: 'UTILISATEUR - NOM OU IDENTIFIANT (u012)' with a placeholder 'Taper nom, prénom ou' and 'LIVRE - TITRE OU AUTEUR' with a placeholder 'Taper titre, auteur o'. A yellow button '+ VALIDER L'EMPRUNT' is below the first field.

Below the form is a statistics bar with four items: '8 EN COURS' (blue), '0 EN RETARD' (red), '6 RETOURNÉS' (green), and '0 PÉNALITÉS FCFA' (red). Below this is a navigation bar with 'EN COURS', 'RETARDS & PÉNALITÉS', 'HISTORIQUE COMPLET', and 'MES EMPRUNTS'.

The main content area shows a list of active loans. Each item has a book cover, title, author, borrower, loan date, return date, and a status 'EN COURS'.

Book Title	Author	Borrower	Loan Date	Return Date	Status
The Hundred-Page Machine Learning Book	Andriy Burkov	Maurice Mathieu	15/05/2026	14/06/2026	EN COURS
Hands-On Machine Learning	Aurélien Géron	Maurice Mathieu	15/05/2026	14/06/2026	EN COURS
Clean Code	Robert C. Martin	Maurice Mathieu	15/05/2026	14/06/2026	EN COURS
Deep Reinforcement Learning Hands-On	Maxim Lapan	Ousmane Ba	13/05/2026	12/06/2026	EN COURS

FIGURE 8 – Section emprunts (vue gestionnaire) — formulaire d’emprunt avec autocomplétion utilisateur/livre et barre de statistiques

=]Pictures/Retour.png

FIGURE 9 – Liste des emprunts actifs — boutons « Valider retour » accessibles exclusivement au gestionnaire

7.6 Gestion des utilisateurs

The screenshot shows the 'Utilisateurs' section of the DIT Bibliothèque application. At the top, there's a navigation bar with 'DIT Bibliothèque' and links for 'LIVRES', 'UTILISATEURS' (highlighted), 'EMPRUNTS', 'DASHBOARD', and 'RECOMMANDATIONS'. On the right, there's a user profile 'DIT Admin' with 'PERSONNEL' and 'DÉCONNEXION' options.

The main heading is 'Utilisateurs' with a subtext '106 utilisateurs'. Below this is a form titled 'Inscrire un nouvel utilisateur'. It has five input fields: 'NOM' (Diallo), 'PRÉNOM' (Mamadou), 'EMAIL' (mamadou@dit.sn), 'MOT DE PASSE' (*****), and 'TYPE' (Étudiant). A yellow button '+ INSCRIRE' is below the first field.

Below the form is a search bar 'Rechercher par initiaux (MD, AS...) ou identifiant (u012)...' and a dropdown 'Tous les types'.

The main content area shows a list of existing users. Each item has a user card with a name, ID, email, status, and buttons for 'EMPRUNTS', '+ EMPRUNT', and 'SUPPRIMER'.

User Name	ID	Email	Status	Buttons
Mathieu Maurice	u104	mathieu@gmail.com	Actif	EMPRUNTS, + EMPRUNT, SUPPRIMER
Konata Aminata	u103	aminatal77855412@dit.sn	Actif	EMPRUNTS, + EMPRUNT, SUPPRIMER
Test Final	u102	final177855647@dit.sn	Actif	EMPRUNTS, + EMPRUNT, SUPPRIMER
Test Claude	u101	test.claude177855371@dit.sn	Actif	EMPRUNTS, + EMPRUNT, SUPPRIMER

FIGURE 10 – Section utilisateurs — 106 inscrits, identifiants courts (u001 à u104), formulaire d’inscription à droite

7.7 Système de recommandation

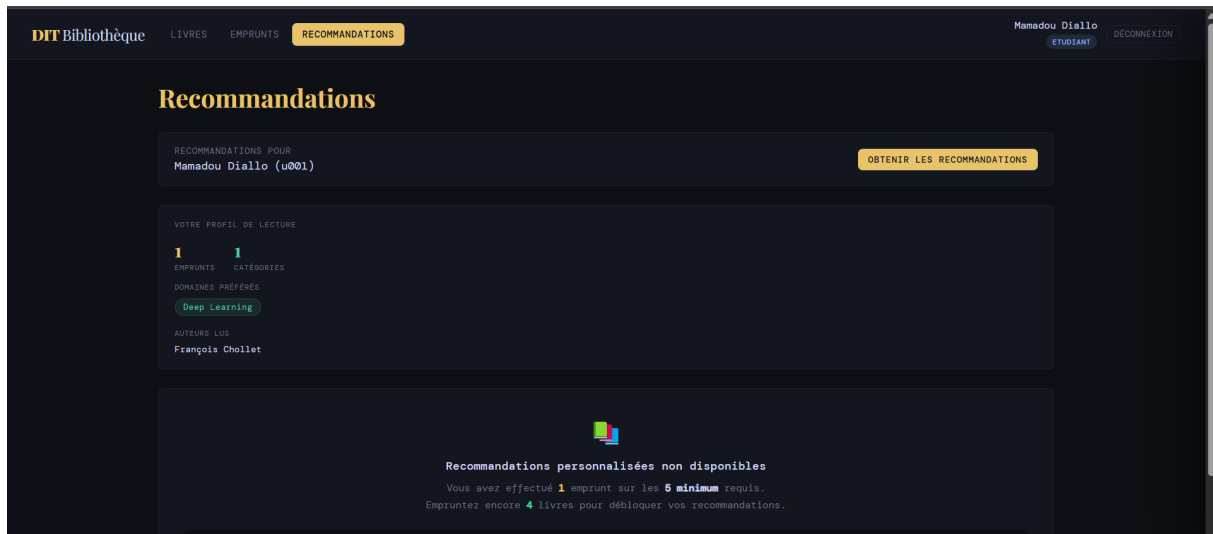


FIGURE 11 — Recommendations — barre de progression (1 emprunt sur 5 requis), catégorie détectée : Deep Learning

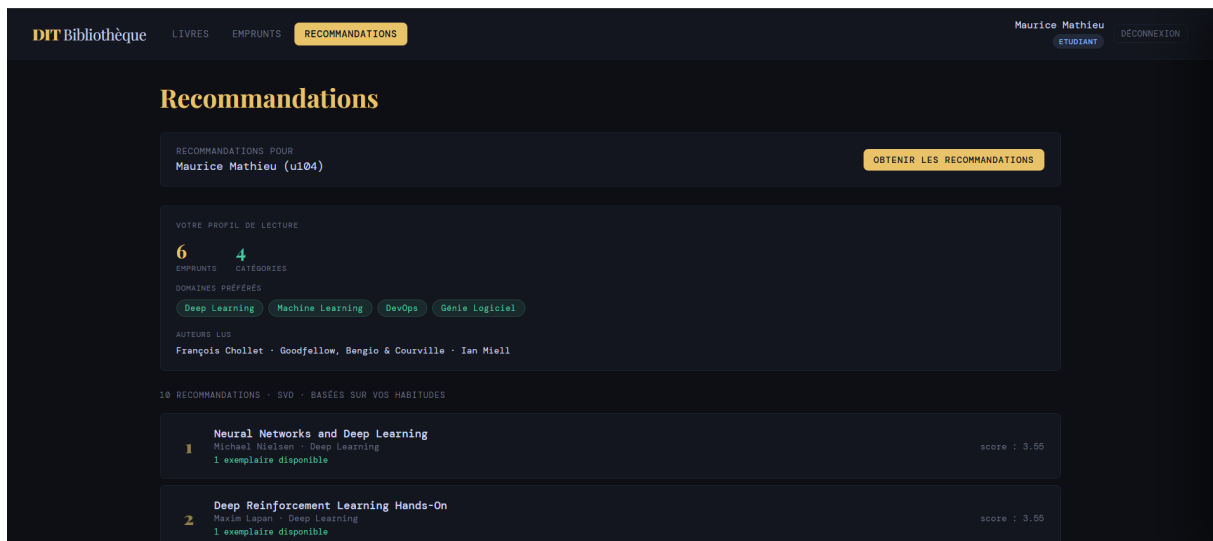


FIGURE 12 — Recommendations SVD actives (6 emprunts) — profil de lecture : Deep Learning, Machine Learning, DevOps, Génie Logiciel ; score : 3.55

Conclusion

La **Bibliothèque Numérique DIT** démontre l'intégration réussie de plusieurs outils de versionnage et de déploiement modernes au service d'une application intelligente :

- **Git** avec une stratégie GitFlow, des branches par microservice et des commits conventionnels pour une traçabilité complète.

- **Docker Compose** avec des profils **dev/prod** orchestrant 7 conteneurs hétérogènes (Node.js, Python, PostgreSQL, nginx).
- **DVC** garantissant la reproductibilité du pipeline ML : export CSV → prétraitement → entraînement SVD → métriques versionnées.
- **Jenkins** automatisant le cycle complet build → test → **dvc repro** → deploy à chaque push.
- **FastAPI** + **scikit-surprise** pour un service de recommandation adaptative avec trois niveaux de réponse selon le profil de l'utilisateur.

En production, la plateforme gère **410 livres**, **106 utilisateurs** et génère des recommandations personnalisées avec un **RMSE de 1.38** sur une échelle de notation de 1 à 5.

Références

- Documentation officielle Docker : <https://docs.docker.com>
- Documentation officielle DVC : <https://dvc.org/doc>
- Documentation scikit-surprise (SVD) : <https://surpriselib.com>
- Documentation FastAPI : <https://fastapi.tiangolo.com>
- Documentation React 18 : <https://react.dev>
- Documentation PostgreSQL 17 : <https://www.postgresql.org/docs/17>
- Atlassian — Gitflow Workflow : <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>
- Conventional Commits : <https://www.conventionalcommits.org>
- Dépôt GitHub du projet : https://github.com/Djidjooh/Bibliotheque_DIT2026